

**ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK**

|                         |                                                                                                                       |                      |                                                   |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------|---------------------------------------------------|
| <b>Course Code:</b>     | CSA06                                                                                                                 | <b>Course Title:</b> | Design and Analysis of Algorithms                 |
| <b>CO Assessed:</b>     | CO2 – Manipulation (BL1, BL2, BL3)                                                                                    | <b>Assessment:</b>   | Assessment Tool 3 – Debugging & Optimisation Task |
| <b>Weightage:</b>       | 10% of CIA                                                                                                            | <b>Total Marks:</b>  | 100                                               |
| <b>CO2 Statement:</b>   | <i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i> |                      |                                                   |
| <b>Student Name:</b>    | G.Srivardhan                                                                                                          | <b>Reg. No.:</b>     | 192425208                                         |
| <b>Section / Batch:</b> |                                                                                                                       | <b>Date:</b>         |                                                   |

**Instructions to Students**

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

| Question Type                            | Focus Area                                                                                      | Questions        | Marks        |
|------------------------------------------|-------------------------------------------------------------------------------------------------|------------------|--------------|
| <b>Debugging &amp; Optimisation Task</b> | Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques | Q1 - Q2          | 100 (2 × 50) |
| <b>GRAND TOTAL:</b>                      |                                                                                                 | <b>100 Marks</b> |              |

**SECTION A – DEBUGGING & OPTIMISATION TASK****Q1. Debugging Convex Hull (Incorrect Orientation Logic)**

The given orientation function incorrectly determines turn direction, affecting convex hull formation. Identify the logical error in the formula. Explain its impact on boundary detection. Debug the function to correctly compute orientation. Optimize the overall hull checking process. Analyze time complexity and rewrite the corrected logic.

```
def orientation(p, q, r):
```

```
    return (q[0]-p[0])*(r[1]-q[1]) - (q[1]-p[1])*(r[0]-q[0]) # ISSUE
```

Debugging Convex Hull (Incorrect Orientation Logic)

a) Identify the Logical Error

The given function is:

```
def orientation(p, q, r):
```

```
    return (q[0]-p[0])*(r[1]-q[1]) - (q[1]-p[1])*(r[0]-q[0]) # ISSUE
```

Logical Error

The function computes the cross product using vectors:

- $PQ = (q - p)$
- $QR = (r - q)$

For a convex hull, the orientation should be computed consistently using the vectors from the same reference point (typically P):

- $PQ = (q - p)$
- $PR = (r - p)$

Using QR instead of PR can lead to incorrect turn detection in convex hull implementations, especially if the implementation assumes the standard orientation formula.

#### b) Impact on Boundary Detection

An incorrect orientation calculation can:

- Misidentify clockwise and counterclockwise turns.
- Include interior points in the convex hull.
- Exclude actual boundary points.
- Produce an incorrect or incomplete convex hull.
- Cause algorithms such as Graham Scan or Jarvis March to fail.

For example, instead of detecting a left turn, the algorithm may detect a right turn and remove a valid hull point.

#### c) Corrected Orientation Function

def orientation(p, q, r):

```
    return ((q[0] - p[0]) * (r[1] - p[1])  
            - (q[1] - p[1]) * (r[0] - p[0]))
```

Or, equivalently, using coordinate expansion:

def orientation(p, q, r):

```
    return ((q[0]-p[0])*(r[1]-p[1]) -  
            (q[1]-p[1])*(r[0]-p[0]))
```

Interpretation

- $> 0 \rightarrow$  Counterclockwise (Left Turn)
- $< 0 \rightarrow$  Clockwise (Right Turn)
- $= 0 \rightarrow$  Collinear

#### d) Optimizing the Hull Checking Process

Instead of checking every point against every other point repeatedly:

- Sort the points once.
- Use Graham Scan or Andrew's Monotonic Chain.
- Maintain a stack of hull vertices.
- Remove points that create clockwise turns.

This reduces unnecessary orientation checks.

#### Better Algorithms

| Algorithm                | Time Complexity |
|--------------------------|-----------------|
| Brute Force Convex Hull  | $O(n^3)$        |
| Jarvis March             | $O(nh)$         |
| Graham Scan              | $O(n \log n)$   |
| Andrew's Monotonic Chain | $O(n \log n)$   |

where  $h$  is the number of hull vertices.

#### e) Time Complexity Analysis

##### Orientation Function

- Arithmetic operations are constant.
- Time Complexity:  $O(1)$
- Space Complexity:  $O(1)$

##### Overall Convex Hull

Using Graham Scan:

- Sorting:  $O(n \log n)$
- Hull construction:  $O(n)$

Total Time Complexity:  $O(n \log n)$

## Corrected Code

```
def orientation(p, q, r):  
    """  
  
    Returns:  
  
    > 0 : Counterclockwise  
  
    < 0 : Clockwise  
  
    = 0 : Collinear  
  
    """  
  
    return ((q[0] - p[0]) * (r[1] - p[1])  
            - (q[1] - p[1]) * (r[0] - p[0]))
```

## Summary

- Logical Error: Orientation was computed using vector QR instead of the standard PR reference, which can conflict with convex hull implementations expecting the conventional cross-product form.
- Impact: Incorrect left/right turn detection, leading to wrong convex hull boundaries.
- Fix: Compute the cross product of vectors PQ and PR.
- Optimized Approach: Use Graham Scan or Andrew's Monotonic Chain for  $O(n \log n)$  performance instead of brute-force methods.

## Q2. Debugging Maximum Element Search Code (Incorrect Handling of Negative-Only Arrays in C)

The following maximum-element function in C initializes the answer to -1, which produces incorrect output when all values in the array are smaller than -1. Carefully analyze why the initialization must depend on actual data values rather than an assumed threshold. Apply systematic debugging so that the maximum element is initialized safely from the array after validating input size. Optimize the corrected implementation for robustness and clear error handling. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```
int findMax(int arr[], int n) {  
    int i, maxVal = -1; // ERROR  
  
    for (i = 0; i < n; i++) {  
        if (arr[i] > maxVal)  
            maxVal = arr[i];  
    }  
  
    return maxVal;  
}
```

Sol:

Debugging Maximum Element Search Code in C

a) Problem Analysis

The given function is:

```
int findMax(int arr[], int n) {  
    int i, maxVal = -1; // ERROR  
    for (i = 0; i < n; i++) {  
        if (arr[i] > maxVal)  
            maxVal = arr[i];  
    }  
    return maxVal;  
}
```

Logical Error

The variable maxVal is initialized to -1, assuming that every array element is greater than or equal to -1.

This assumption is incorrect.

Example

Array = {-8, -15, -3, -20}

Execution:

- maxVal = -1
- -8 > -1 → False
- -15 > -1 → False
- -3 > -1 → False
- -20 > -1 → False

Returned value:

-1

Incorrect, because -1 is not even present in the array.

The correct maximum is:

-3

Therefore, the maximum should be initialized using an actual array element, not a fixed value.

## b) Correct Debugging Approach

1. Validate the array size ( $n > 0$ ).
2. Initialize the maximum with the first array element.
3. Compare remaining elements.
4. Return the maximum element.

This works correctly for:

- Positive numbers
- Negative numbers
- Mixed values
- Single-element arrays

## c) Corrected and Robust C Code

```
#include <stdio.h>
```

```
/* Function to find the maximum element in an array */
```

```
int findMax(int arr[], int n)
```

```
{  
    // Check for invalid array size  
    if (n <= 0)  
    {  
        printf("Error: Array is empty.\n");  
        return 0; // Error indicator  
    }
```

```
    // Initialize maximum with the first element
```

```
    int maxVal = arr[0];
```

```
    // Compare remaining elements
```

```
    for (int i = 1; i < n; i++)
```

```
{
```

```

        if (arr[i] > maxVal)
        {
            maxVal = arr[i];
        }
    }

    return maxVal;
}

int main()
{
    int arr[] = {-8, -15, -3, -20};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Maximum element = %d\n", findMax(arr, n));

    return 0;
}

```

#### Sample Output

Maximum element = -3

#### d) Why This Solution Is Robust

- ✓ Handles arrays containing only negative numbers.
- ✓ Works for positive and mixed-value arrays.
- ✓ Correctly processes single-element arrays.
- ✓ Validates the input size before accessing the array.
- ✓ Uses constant extra memory.

#### e) Time Complexity Analysis

Let  $n$  be the number of elements.

Operation      Complexity

Input validation  $O(1)$

Initialization       $O(1)$

Single traversal  $O(n)$

Overall Time Complexity:  $O(n)$

Each element is examined exactly once.

Space Complexity:  $O(1)$

Only one extra variable ( $\text{maxVal}$ ) is used.

#### Summary

- Bug: Initializing  $\text{maxVal}$  to  $-1$  fails for arrays where all elements are less than  $-1$ .
- Fix: Initialize  $\text{maxVal}$  with the first array element after verifying that the array is not empty.
- Time Complexity:  $O(n)$
- Space Complexity:  $O(1)$
- Advantage: The corrected implementation is accurate, robust, and works for all valid integer arrays, including those containing only negative values.

#### Code of Conduct

*I G.Srivardan(192425208) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: \_\_\_\_\_

#### RUBRICS FOR CALCULATION

| Criteria               | Wt. | Level 4 – Exemplary                                                       | Level 3 – Proficient                   | Level 2 – Developing                          | Level 1 – Beginning                 |
|------------------------|-----|---------------------------------------------------------------------------|----------------------------------------|-----------------------------------------------|-------------------------------------|
| Problem Identification | 20% | Accurately identifies all errors and root causes with clear understanding | Identifies most errors with minor gaps | Identifies some errors but misses key issues  | Unable to identify errors correctly |
| Debugging              | 25% | Applies systematic debugging techniques                                   | Uses appropriate debugging             | Limited debugging approach; requires guidance | Unable to debug or resolve issues   |



|              |     |                                                                                      |                                                        |                                               |                                               |
|--------------|-----|--------------------------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------|-----------------------------------------------|
|              |     | effectively to resolve issues                                                        | methods with minor errors                              |                                               |                                               |
| Optimization | 20% | Optimizes code by significantly improving time complexity using efficient algorithms | Improves time complexity with minor gaps in efficiency | Limited improvement in time complexity        | No improvement; inefficient approach retained |
| Analysis     | 20% | Demonstrates strong logical reasoning and clear justification of solutions           | Shows reasonable analysis with some justification      | Limited analysis; weak reasoning              | No analytical reasoning demonstrated          |
| Code Quality | 15% | Produces clean, well-structured code with proper comments and documentation          | Code is mostly clear with minor documentation gaps     | Code lacks structure and proper documentation | Poorly written code with no documentation     |

### Marks Evaluation:

| Question No.        | Problem Identification (20%) | Debugging (25%) | Optimization (20%) | Analysis (20%) | Code Quality (15%) | Total Marks (Each – 50) |
|---------------------|------------------------------|-----------------|--------------------|----------------|--------------------|-------------------------|
| Q1                  | 4                            | 3               | 3                  | 2              | 4                  | 39                      |
| Q2                  | 4                            | 4               | 4                  | 3              | 2                  | 44                      |
| Overall Total (100) |                              |                 |                    |                |                    | 83                      |

| Faculty In-charge | Course Coordinator | Head of Department |
|-------------------|--------------------|--------------------|
|                   |                    |                    |
| Signature: _____  | Signature: _____   | Signature: _____   |
| Date: _____       | Date: _____        | Date: _____        |